Project 2: State Feedback Control

24-677 Special Topics: Linear Control Systems

Prof. M. Bedillion
Based on materials originally developed by Prof. Ding Zhao.

Due: Nov 06, 2026, 11:59 pm.

- We will use Gradescope to grade. The link is on the panel of CANVAS.
- Submit **your_controller.m** to Gradescope under **Programming-P2** and your solutions in PDF format to **Project-P2**. Insert `controller_results.png` in the PDF. We will test **your_controller.m** and manually check the written answers.
- This repository targets **Webots R2025a**. Install that release from the official Webots releases page; using an older release may rewrite the world in an incompatible format.
- The controller backend is entirely MATLAB; Python, NumPy, SciPy, and Matplotlib are not required. The project has been verified with 64-bit **MATLAB R2025a**. On Windows, also install the MATLAB Support for MinGW-w64 C/C++ Compiler add-on, which Webots needs to load its controller and vehicle-driver libraries. NOTE: We have had mixed results with Matlab 2026a. If you are having "Load Library" issues, we recommend installing 2025a for a seamless experience.
- If Webots does not detect MATLAB, set the MATLAB command under **Tools** → **Preferences** → **General**. On Windows, select `MATLAB.exe` under `bin\win64` (for example, `C:\Program Files\MATLAB\R2025a\bin\win64\MATLAB.exe`), not the launcher under `bin`.
- Please familiarize yourself with Webots documentation, specifically their User Guide and their Webots for Automobiles section, if you encounter difficulties in setup or use. It will help to have a good understanding of the underlying tools that will be used in this assignment. To that end, completing at least Tutorial 1 in the user guide is highly recommended. **Please do not post your code implementation publicly**
- Open `worlds/automotive_new.wbt` from Webots and press Run. Do not start `main_matlab.m` directly: Webots must establish the controller connection and environment first.
- If you have issues with Webots that are beyond the scope of the documentation (e.g. the software runs too slow, crashes, or has other odd behavior), please let us know via Piazza. We will do our best to help.

- We advise you to start with the assignment early.

1 Introduction

This project will allow you to practice the following skills

- Checking controllability and observability
- Design a state feedback controller
- Design a full state observer

As in Project 1, this project consists of a written component along with a Webots controller design component.

2 Model

State feedback control is used to drive the states to the origin, but in this case we are following a trajectory. For this project we will follow the lead of [1] and redefine the state space system developed in Project 1 to be based on the error in lateral position. Like Project 1, we will assume that the dynamics are decoupled between the lateral and longitudinal directions, which allows us to independently design lateral and longitudinal controllers.

We will define the following states to capture the error (see Section 2.5 in [1] for more detail):

- e_1 - the distance of the vehicle mass center from the track centerline. Note that this was absent in the first Project!
- e_2 - the orientation of the vehicle with respect to the road. This is what we controlled with a PID controller in Project 1.

Using state feedback we will be able to account for BOTH position errors relative to the center and the heading - this might make state feedback more successful than PID control for this system.

Assume that the longitudinal velocity V_x is constant and the radius of the road R is constant and large. Then we can define the desired rate of orientation change as

$$\dot{\psi}_{des} = \frac{V_x}{R}.$$

The desired acceleration is then given by

$$a_{lat_{des}} = \frac{V_x^2}{R} = V_x \dot{\psi}_{des}.$$

Now we can compute the error in lateral acceleration

$$\ddot{e}_1 = \underbrace{\ddot{y} + V_x \dot{\psi}}_{a_{lat}} - \underbrace{\frac{V_x^2}{R}}_{a_{lat_{des}}} = \ddot{y} + V_x \left(\dot{\psi} - \dot{\psi}_{des} \right)$$

With V_x and a constant we can integrate this easily

$$\dot{e}_1 = \dot{y} + V_x \left(\psi - \psi_{des}\right) \tag{1}$$

Using the definition of the heading error ($e_2 = \psi - \psi_{des}$) along with the dynamics in Project 1 we can derive the following error dynamics equation.

$$\frac{d}{dt}\begin{bmatrix} e_1 \\ \dot{e}_1 \\ e_2 \\ \dot{e}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{4C_\alpha}{m\dot{x}} & \frac{4C_\alpha}{m} & -\frac{2C_\alpha(l_f - l_r)}{m\dot{x}} \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{2C_\alpha(l_f - l_r)}{I_z\dot{x}} & \frac{2C_\alpha(l_f - l_r)}{I_z} & -\frac{2C_\alpha(l_f^2 + l_r^2)}{I_z\dot{x}} \end{bmatrix}\begin{bmatrix} e_1 \\ \dot{e}_1 \\ e_2 \\ \dot{e}_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ \frac{2C_\alpha}{m} & 0 \\ 0 & 0 \\ \frac{2C_\alpha l_f}{I_z} & 0 \end{bmatrix}\begin{bmatrix} \delta \\ F \end{bmatrix} + \begin{bmatrix} o \\ -\frac{2C_\alpha(l_f - l_r)}{m\dot{x}} - \dot{x} \\ 0 \\ -\frac{2C_\alpha(l_f^2 + l_r^2)}{I_z\dot{x}} \end{bmatrix}\dot{\psi}_{des}$$

In lateral vehicle dynamics, $\dot{\psi}_{des}$ is a time-varying disturbance in the state space equation. Its value is proportional to the longitudinal speed when the radius of the road is constant. When deriving the error-based state space model for controller design, $\dot{\psi}_{des}$ can be safely assumed to be zero.

$$\frac{d}{dt}\begin{bmatrix} e_1 \\ \dot{e}_1 \\ e_2 \\ \dot{e}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{4C_\alpha}{m\dot{x}} & \frac{4C_\alpha}{m} & -\frac{2C_\alpha(l_f - l_r)}{m\dot{x}} \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{2C_\alpha(l_f - l_r)}{I_z\dot{x}} & \frac{2C_\alpha(l_f - l_r)}{I_z} & -\frac{2C_\alpha(l_f^2 + l_r^2)}{I_z\dot{x}} \end{bmatrix}\begin{bmatrix} e_1 \\ \dot{e}_1 \\ e_2 \\ \dot{e}_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ \frac{2C_\alpha}{m} & 0 \\ 0 & 0 \\ \frac{2C_\alpha l_f}{I_z} & 0 \end{bmatrix}\begin{bmatrix} \delta \\ F \end{bmatrix}$$

For the longitudinal control:

$$\frac{d}{dt}\begin{bmatrix} x \\ \dot{x} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}\begin{bmatrix} x \\ \dot{x} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{m} \end{bmatrix}\begin{bmatrix} \delta \\ F \end{bmatrix} + \begin{bmatrix} 0 \\ \dot{\psi}\dot{y} - fg \end{bmatrix}$$

Assuming $\dot{\psi} = 0$:

$$\frac{d}{dt}\begin{bmatrix} x \\ \dot{x} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}\begin{bmatrix} x \\ \dot{x} \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{m} \end{bmatrix}\begin{bmatrix} \delta \\ F \end{bmatrix}$$

On the sensing side, we typically know the location of the car (GPS) with its heading (compass) and longitudinal velocity (tachometer). In other words, we cannot directly measure the full state in the error dynamics model.

3 Project 2: State Space Control [Due Nov 6, 2026]

Exercise 1. To warm up for the exercises in the project (and help prep for the 2nd exam), here are several typical homework problems on controllability, observability, controller design, and observer design.

1. For the following systems determine controllability and observability.

   - $\dot{x} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & -3 & -3 \end{bmatrix} x + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} u,\ y = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} x$
   - $\dot{x} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 2 & -1 \end{bmatrix} x + \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} u,\ y = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix} x$
   - $\dot{x} = \begin{bmatrix} 2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} x + \begin{bmatrix} 2 & 1 & 0 \\ 2 & 1 & 1 \\ 1 & 1 & 1 \\ 3 & 2 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} u,\ y = \begin{bmatrix} 2 & 2 & 1 & 3 & -1 & 1 & 1 \\ 1 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix} x$

2. Find a state space realization for $G(s) = \begin{bmatrix} \frac{1}{s} & \frac{s+3}{s+1} \\ \frac{1}{s+3} & \frac{s}{s+1} \end{bmatrix}$.

3. Consider the discrete time system $x_{k+1} = \begin{bmatrix} 1 & 1 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} u_k$. Design a state feedback control matrix K such that the closed loop system has all poles at 0.

4. Consider the discrete time system $x_{k+1} = \begin{bmatrix} -2 & 4 \\ -3 & 9 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u,\ y = \begin{bmatrix} 3 & 1 \end{bmatrix} x$. Design an observer matrix L such that the observer has poles at $-0.5 \pm 0.5j$.

Exercise 2. Considering the linearized, error-based state space system for the vehicle in the Model section above:

1. Check the controllability of the system at the following longitudinal velocities: 2 m/s, 5 m/s and 8 m/s.

2. For longitudinal velocities v from 1 m/s to 40 m/s, plot the following:

   (a) $log_{10}(\frac{\sigma_1}{\sigma_n})$ versus v (m/s), where σ_i is the ith singular value of the controllability matrix P ($i = 1, 2, ..., n$). (In other words, what is the logarithm of the greatest singular value divided by the smallest?)

(b) $Re(p_i)$ versus v (m/s), where Re is real part and p_i is the ith pole of the continuous state space system. [Use 4 subplots, one for each of the 4 poles]

What conclusions can you draw about the overall controllability and stability of the system in observing these two plots?

3. Assuming that we can directly measure both lateral error terms but not their derivatives, check the observability of the lateral error dynamics at the following longitudinal velocities: 2 m/s, 5 m/s and 8 m/s. If we measure ONLY the heading error (as in Project 1) what is the conclusion on observability?

4. Show your design for the pole placement controller that you used in Exercise 3. How did you select your desired pole locations?

5. Given measurements of only the two error terms, how would you reconstruct the error derivative states to enable your state feedback controller?

Submit you answers in the **.pdf** file and also submit the Matlab live script you used to generate it.

Exercise 3. For the lateral control of the vehicle, design a state feedback controller using pole placement. Tune the poles of the closed loop system such that it can achieve the performance criteria mentioned below.

You can reuse your longitudinal PID controller from part 1 of this project, or even improve upon it. However, it may require retuning based on observed performance.

Design the two controllers in `your_controller.m`. You can make use of Webots' built-in code editor, or use your own.

Check the performance of your controller by running the Webots simulation. You can press the play button in the top menu to start the simulation in real-time, the fast-forward button to run the simulation as quickly as possible, and the triple fast-forward to run the simulation without rendering (any of these options is acceptable, and the faster options may be better for quick tests). If you complete the track, the scripts will generate a performance plot. This plot contains a visualization of the car's trajectory, and also shows the variation of states with respect to time.

Submit `your_controller.m` and the final completion plot as described on the title page. Your controller is **required** to achieve the following performance criteria to receive full points:

1. Time to complete the loop = 350 s

2. Maximum deviation from the reference trajectory = 8.0 m

3. Average deviation from the reference trajectory = 4.0 m

Some hints that may be useful:

- It is somewhat difficult to tune pole-placement controllers. Learning optimal control in the next submodule will fortunately make this task much easier. Some tips to help for this assignment follow.
  - Poles must be negative if the system is stable.
  - Poles can be complex, where an imaginary number is denoted with `j`, e.g. `-3+1j`. If you use a complex pole, you must also include its complex conjugate.
  - Don't use the poles from Exercise 1 as a starting point - these are the system's open-loop poles. Your goal is to select new positions for the closed-loop poles.
  - Poles placed closer to the imaginary axis (in other words, closer to 0 on the real axis) will dominate the system response. Recall that the system responds faster the farther the poles are from the imaginary axis (provided they are stable).
  - Having at least one dominant pole to help the system to converge is recommended. The placement of your other poles is up to you based on your performance. Alternatively, you can also place a pair of conjugate poles closer to the imaginary axis, and keep the other two away from it.
- The controller itself can be continuous or discrete - it is your choice whether to discretize the system or not.
- You need not design and implement an observer for this problem.

4 Appendix

(Already covered in Project 1)

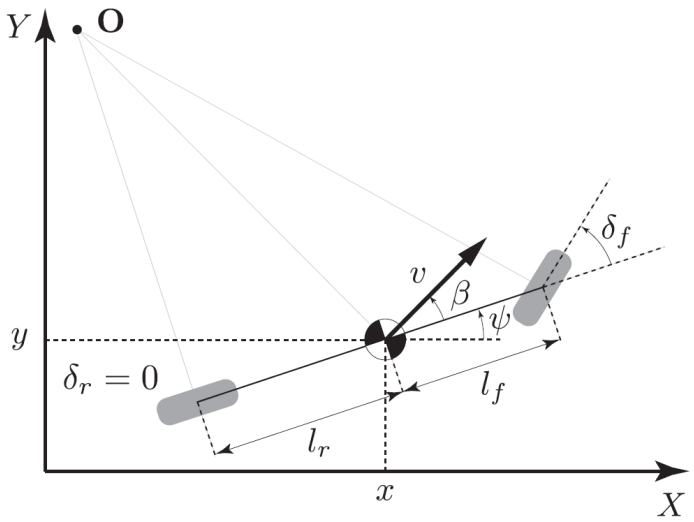


Figure 1: Bicycle model[2]

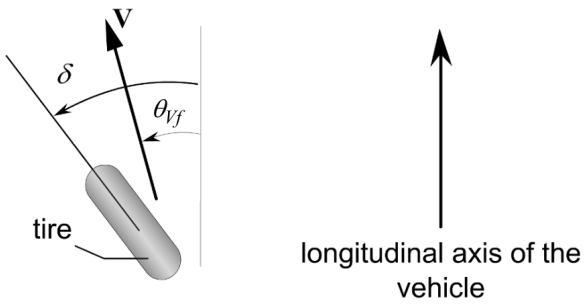


Figure 2: Tire slip-angle[2]

We will make use of a bicycle model for the vehicle, which is a popular model in the study of vehicle dynamics. Shown in Figure 1, the car is modeled as a two-wheel vehicle with two degrees of freedom, described separately in longitudinal and lateral dynamics. The model parameters are defined in Table 2.

4.1 Lateral dynamics

Ignoring road bank angle and applying Newton's second law of motion along the y-axis:

$$ma_y = F_{yf}\cos\delta_f + F_{yr}$$

where $a_y = \left(\frac{d^2y}{dt^2}\right)_{inertial}$ is the inertial acceleration of the vehicle at the center of geometry in the direction of the y axis, F_{yf} and F_{yr} are the lateral tire forces of the front and rear wheels, respectively, and δ_f is the front wheel angle, which will be denoted as δ later. Two terms contribute to a_y: the acceleration $\ddot{y}$, which is due to motion along the y-axis, and the centripetal acceleration. Hence:

$$a_y = \ddot{y} + \dot{\psi}\dot{x}$$

Combining the two equations, the equation for the lateral translational motion of the vehicle is obtained as:

$$\ddot{y} = -\dot{\psi}\dot{x} + \frac{1}{m}(F_{yf}\cos\delta + F_{yr})$$

Moment balance about the axis yields the equation for the yaw dynamics as

$$\ddot{\psi}I_z = l_f F_{yf} - l_r F_{yr}$$

The next step is to model the lateral tire forces F_{yf} and F_{yr}. Experimental results show that the lateral tire force of a tire is proportional to the "slip-angle" for small slip-angles when vehicle's speed is large enough - i.e. when $\dot{x} \geq 0.5$ m/s. The slip angle of a tire is defined as the angle between the orientation of the tire and the orientation of the velocity vector of the vehicle. The slip angle of the front and rear wheel is

$$\alpha_f = \delta - \theta_{Vf}$$
$$\alpha_r = -\theta_{Vr}$$

where θ_{Vp} is the angle between the velocity vector and the longitudinal axis of the vehicle, for $p \in \{f, r\}$. A linear approximation of the tire forces are given by

$$F_{yf} = 2C_\alpha \left(\delta - \frac{\dot{y} + l_f \dot{\psi}}{\dot{x}} \right)$$
$$F_{yr} = 2C_\alpha \left(-\frac{\dot{y} - l_r \dot{\psi}}{\dot{x}} \right)$$

where C_α is called the cornering stiffness of the tires. If $\dot{x} < 0.5$ m/s, we just set F_{yf} and F_{yr} both to zeros.

4.2 Longitudinal dynamics

Similarly, a force balance along the vehicle longitudinal axis yields:

$$\ddot{x} = \dot{\psi}\dot{y} + a_x$$
$$m a_x = F - F_f$$
$$F_f = fmg$$

where F is the total tire force along the x-axis, and F_f is the force due to rolling resistance at the tires, and f is the friction coefficient.

4.3 Global coordinates

In the global frame we have:

$$\dot{X} = \dot{x}\cos\psi - \dot{y}\sin\psi$$
$$\dot{Y} = \dot{x}\sin\psi + \dot{y}\cos\psi$$

4.4 System equation

Gathering all of the equations, if $\dot{x} \geq 0.5$ m/s, we have:

$$\ddot{y} = -\dot{\psi}\dot{x} + \frac{2C_\alpha}{m}(\cos\delta \left(\delta - \frac{\dot{y} + l_f \dot{\psi}}{\dot{x}} \right) - \frac{\dot{y} - l_r \dot{\psi}}{\dot{x}})$$
$$\ddot{x} = \dot{\psi}\dot{y} + \frac{1}{m}(F - fmg)$$
$$\ddot{\psi} = \frac{2l_f C_\alpha}{I_z} \left(\delta - \frac{\dot{y} + l_f \dot{\psi}}{\dot{x}} \right) - \frac{2l_r C_\alpha}{I_z} \left(-\frac{\dot{y} - l_r \dot{\psi}}{\dot{x}} \right)$$
$$\dot{X} = \dot{x}\cos\psi - \dot{y}\sin\psi$$
$$\dot{Y} = \dot{x}\sin\psi + \dot{y}\cos\psi$$

otherwise, since the lateral tire forces are zeros, we only consider the longitudinal model.

4.5 Measurements

The observable states are:

$$y = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \\ X \\ Y \\ \psi \end{bmatrix}$$

4.6 Physical constraints

The system satisfies the constraints that:

$$|\delta| \leqslant \tfrac{\pi}{6}\ rad$$
$$F \geqslant 0 \text{ and } F \leqslant 16000\ N$$
$$\dot{x} \geqslant 10^{-5}\ m/s$$

Table 1: Model parameters.

Name	Description	Unit	Value
$(\dot{x}, \dot{y})$	Vehicle's velocity along the direction of vehicle frame	m/s	State
(X, Y)	Vehicle's coordinates in the world frame	m	State
ψ, $\dot{\psi}$	Body yaw angle, angular speed	rad, rad/s	State
δ or δ_f	Front wheel angle	rad	Input
F	Total input force	N	Input
m	Vehicle mass	kg	1000
l_r	Length from rear tire to the center of mass	m	0.82
l_f	Length from front tire to the center of mass	m	1.18
C_α	Cornering stiffness of each tire	N	20000
I_z	Yaw inertia	kg m^2	3004.5
F_{pq}	Tire force, $p \in \{x, y\}$,$q \in \{f, r\}$	N	Depends on input force
f	Rolling resistance coefficient	N/A	0.028
`delT`	Simulation timestep	sec	0.032

4.7 Simulation

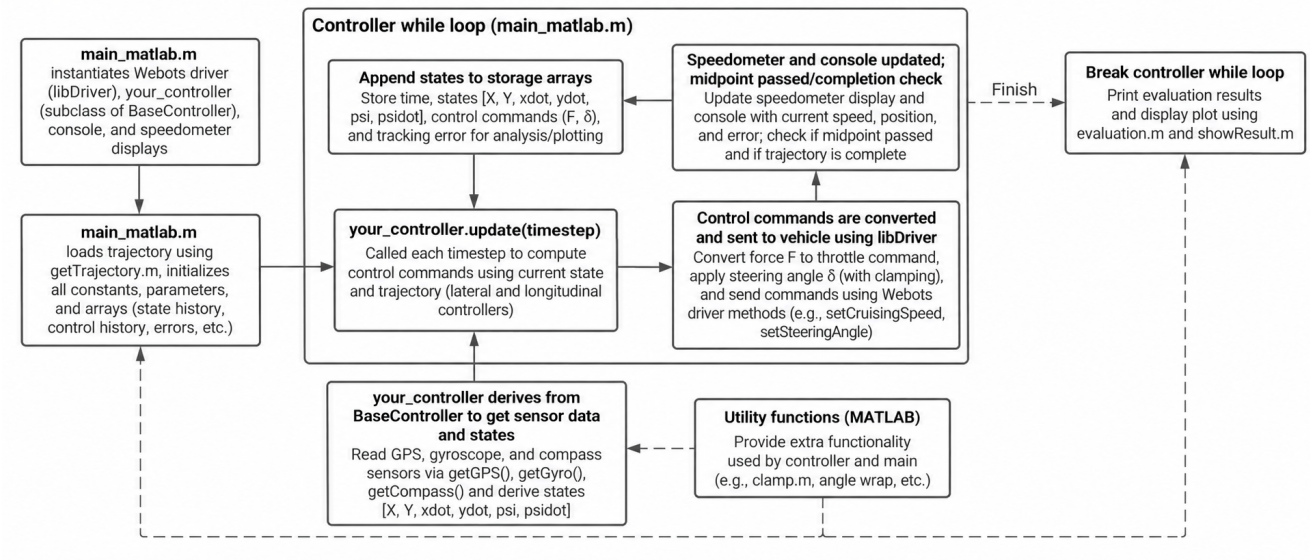


Figure 3: Simulation code flow

Several files are provided to you within the `controllers/main` folder. The `main.m` script initializes and instantiates necessary objects, and also contains the controller loop. This loop runs once each simulation timestep. `main.m` calls `your_controller.m`'s `update` method on each loop to get new control commands (the desired steering angle, δ, and longitudinal force, F). The longitudinal force is converted to a throttle input, and then both control commands are set by Webots internal functions. The additional script `util.m` contains functions to help you design and execute the controller. The full codeflow is pictured in Figure 3.

Please design your controller in the `your_controller.m` file provided for the project part you're working on. Specifically, you should be writing code in the `update` method. Please **do not** attempt to change code in other functions or files, as we will only grade the relevant `your_controller.m` for the programming portion. However, you are free to add to the `CustomController` class's `__init__` method (which is executed once when the `CustomController` object is instantiated).

4.8 BaseController Background

The `CustomController` class within each `your_controller.m` file derives from the `BaseController` class in the `base_controller.m` file. The vehicle itself is equipped with a Webots-generated GPS, gyroscope, and compass that have no noise or error. These sensors are started in the `BaseController` class, and are used to derive the various states of the vehicle. An explanation on the derivation of each can be found in the table below.

Table 2: State Derivation.

Name	Explanation
(X, Y)	From GPS readings
$(\dot{x}, \dot{y})$	From the derivative of GPS readings
ψ	From the compass readings
$\dot{\psi}$	From the gyroscope readings

4.9 Trajectory Data

The trajectory is given in `buggyTrace.csv`. It contains the coordinates of the trajectory as (x, y). The satellite map of the track is shown in Figure 4.

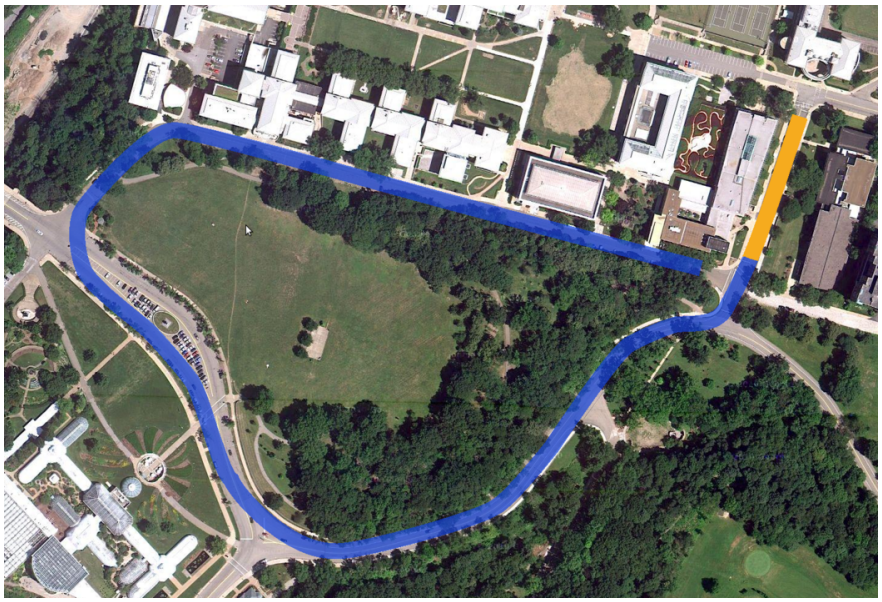


Figure 4: Buggy track[3]

5 Reference

1. Rajamani Rajesh. Vehicle Dynamics and Control. Springer Science & Business Media, 2011.

2. Kong Jason, et al. "Kinematic and dynamic vehicle models for autonomous driving control design." Intelligent Vehicles Symposium, 2015.

3. cmubuggy.org, https://cmubuggy.org/reference/File:Course_hill1.png

4. "PID Controller—Manual Tuning." *Wikipedia*. https://en.wikipedia.org/wiki/PID_controller#Manual_tuning

5. Cyberbotics. "Webots R2025a Release." https://github.com/cyberbotics/webots/releases/tag/R2025a

6. Cyberbotics. "Using MATLAB with Webots." https://cyberbotics.com/doc/guide/using-matlab

7. MathWorks. "MATLAB Support for MinGW-w64 C/C++ Compiler." https://www.mathworks.com/matlabcentral/fileexchange/52848-matlab-support-for-mingw-w64-c-c-com